

LINQ All Method in C# with Examples

In this article, I will discuss the **LINQ All Method in C#** with Examples. Please read our previous article, discussing the basics of [LINQ Quantifiers Methods in C#](#). As part of this article, we are going to discuss the following pointers, which are related to the LINQ All Method.

1. [What is LINQ All Method in C#?](#)
2. [Example to Understand LINQ All Method in C# using Value Type](#)
3. [Example to Understand LINQ All Method in C# using String Type](#)
4. [LINQ All Method with Complex Type in C#](#)
5. [Complex Example to Understand LINQ All Method in C#](#)
6. [When should you use the LINQ All Method in C#?](#)

What is LINQ All Method in C#?

The All method in LINQ (Language Integrated Query) is a standard query operator that checks whether all elements in a collection satisfy a given condition. It is a part of the System.Linq namespace in .NET and can be used with arrays, lists, or any collection that implements the IEnumerable or IQueryable interface. The All method returns a boolean value: true if every element in the sequence satisfies the condition specified by the provided predicate or false if at least one element does not.

There is no overloaded version available for the LINQ All Method. The image below shows that the ALL Extension method takes one predicate as a parameter.

```
public static bool All<TSource>(this IEnumerable<TSource> source, Func<TSource, bool> predicate);
```

Here,

- **TSource:** The collection to check, such as an array or a list.
- **Predicate:** A lambda expression that defines the condition each element must meet. The element variable represents each item in the collection as the method iterates through it.

Example to Understand LINQ All Method in C# using Value Type

Let us see an example of understanding LINQ All Method in C# using Method and Query Syntax. The following example returns true as all the elements are greater than 10 in the integer array. No operator is called all in Query Syntax, so we need to use Mixed Syntax.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

        int[] IntArray = { 11, 22, 33, 44, 55 };
        //Using Method Syntax
        bool ResultMS = IntArray.All(x => x > 10);
        //Using Query Syntax
        bool ResultQS = (from num in IntArray
                        select num).All(x => x > 10);
        Console.WriteLine("Are All Numbers greater than 10? " + ResultMS);
        Console.ReadKey();
    }
}

```

Output: **Are All Numbers greater than 10? True**

How Does the All Method Work in C#?

- **Predicate Function:** The All method requires a predicate function as its parameter. This function takes an element of the collection as input and returns a boolean value indicating whether the element satisfies a specific condition.
- **Iteration:** Internally, the All method iterates over each collection element.
- **Evaluation:** The All method invokes the predicate function for each element, passing the current element as the argument.
- **Condition Check:**
 1. If the predicate function returns false for any element, the All method immediately stops processing and returns false.
 2. If the predicate function returns true for all elements in the collection, the iteration completes, and the All method returns true.

Example to Understand LINQ All Method in C# using String Type

Let us see an example of using the LINQ All Method in C# using String type collection. For a better understanding, please look at the following example, which shows how to use All Method in C# with String type collection. The following example will return false as all the names are not less than five characters.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] stringArray = { "James", "Sachin", "Sourav", "Pam", "Sara" };
            //Using Method Syntax
            bool ResultMS = stringArray.All(name => name.Length > 5);
            //Using Query Syntax
            bool ResultQS = (from num in stringArray
                            select num).All(name => name.Length > 5);
            Console.WriteLine("Are All Names greater than 5 Characters: " + ResultMS);
            Console.ReadKey();
        }
    }
}

```

```

    }
}
}

```

Output: Are All Names greater than 5 Characters: False

Note: We need to use the All method to ensure that every element in the collection must meet the given criteria. All will return false if even one element does not satisfy the condition. It's often used for validation checks or to ensure that a collection only contains items with certain attributes.

Example to Understand LINQ All Method with Complex Type in C#:

Let us see an example of using the LINQ All Method with Complex Data Type in C# using both Method and Query Syntax. We are going to work with the following Student and Subject classes. So, create a class file with the name **Student.cs** and then copy and paste the following code. As you can see, the Student class has four properties: ID, Name, TotalMarks, and Subjects. Here, within the Student class, we have also created one method, i.e., GetAllStudnets(), which will return the list of all the students. The Subject class has only two properties, i.e., SubjectName and Marks.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int TotalMarks { get; set; }
        public List<Subject> Subjects { get; set; }
        public static List<Student> GetAllStudnets()
        {
            List<Student> listStudents = new List<Student>()
            {
                new Student{ID= 101,Name = "Preety", TotalMarks = 265,
                    Subjects = new List<Subject>()
                    {
                        new Subject(){SubjectName = "Math", Marks = 80},
                        new Subject(){SubjectName = "Science", Marks = 90},
                        new Subject(){SubjectName = "English", Marks = 95}
                    }
                },
                new Student{ID= 102,Name = "Priyanka", TotalMarks = 278,
                    Subjects = new List<Subject>()
                    {
                        new Subject(){SubjectName = "Math", Marks = 90},
                        new Subject(){SubjectName = "Science", Marks = 95},
                        new Subject(){SubjectName = "English", Marks = 93}
                    }
                },
                new Student{ID= 103,Name = "James", TotalMarks = 240,
                    Subjects = new List<Subject>()
                    {
                        new Subject(){SubjectName = "Math", Marks = 70},

```

```

        new Subject(){SubjectName = "Science", Marks = 80},
        new Subject(){SubjectName = "English", Marks = 90}
    }},
    new Student{ID= 104, Name = "Hina", TotalMarks = 275,
    Subjects = new List<Subject>()
    {
        new Subject(){SubjectName = "Math", Marks = 90},
        new Subject(){SubjectName = "Science", Marks = 90},
        new Subject(){SubjectName = "English", Marks = 95}
    }},
    new Student{ID= 105, Name = "Anurag", TotalMarks = 255,
    Subjects = new List<Subject>()
    {
        new Subject(){SubjectName = "Math", Marks = 80},
        new Subject(){SubjectName = "Science", Marks = 90},
        new Subject(){SubjectName = "English", Marks = 85}
    }
    },
};
return listStudents;
}
}
public class Subject
{
    public string SubjectName { get; set; }
    public int Marks { get; set; }
}
}

```

Now, our requirement is to check whether all the students have total marks greater than 250. As you can see, the student James's total mark is 240, which is less than 250. So here, the LINQ ALL method will give you the output as false. This is because the All method will return true when all the elements in the collection satisfy the given condition.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            bool MSResult = Student.GetAllStudnets().All(std => std.TotalMarks > 250);
            //Using Query Syntax
            bool QSResult = (from std in Student.GetAllStudnets()
                             select std).All(std => std.TotalMarks > 250);
            Console.WriteLine($"Is All Students Having Total Marks 250: {MSResult}");
            Console.ReadKey();
        }
    }
}

```

```
}
```

Output: Is All Students Having Total Marks 250: False

Note: When used with an IQueryable data source, such as a database context in Entity Framework, the All method translates into a SQL query that executes in the database engine. This can be very efficient for checking conditions against large datasets without loading them into memory.

Complex Example to Understand LINQ All Method in C#:

Let us see a more Complex Example to Understand the LINQ All Method in C#. If you see our student's collection, you will observe that each student object has another collection called Subjects. Now we need to fetch all the student details whose mark on each subject exceeds 80. That means we will not apply the LINQ All method to the student's collection. Rather, we will apply the LINQ All method to the Subject collection of each student.

For a better understanding, please have a look at the following example. The Where Extension method takes a predicate as a parameter, returning a boolean true and false. Boolean TRUE means the element will return, and False means the record will not return. We are applying the LINQ All method on the Subject property within the Where Extension method. Now, for each student, the LINQ All method will execute, and it will check whether all the Subject Marks satisfied the given condition, i.e., Marks > 80, and if satisfied, the All Method will return True, and Where extension method will return that Student in output.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSResult = Student.GetAllStudnets()
                .Where(std => std.Subjects.All(x => x.Marks > 80));
            //Using Query Syntax
            var QSResult = (from std in Student.GetAllStudnets()
                           where std.Subjects.All(x => x.Marks > 80)
                           select std).ToList();
            foreach (var student in QSResult)
            {
                Console.WriteLine($"{student.Name} - {student.TotalMarks}");
                foreach (var subject in student.Subjects)
                {
                    Console.WriteLine($" {subject.SubjectName} - {subject.Marks}");
                }
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Priyanka - 278
  Math - 90
  Science - 95
  English - 93
Hina - 275
  Math - 90
  Science - 90
  English - 95
```

When should you use the LINQ All Method in C#?

The All method in LINQ should be used when you need to verify that all collection elements satisfy a specific condition. Here are some scenarios where the All method is particularly useful:

Checking User Permissions

Suppose you have a collection of user roles and want to verify that all users have administrative access before performing a sensitive operation. Using the All method, you can easily check this condition.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var users = new List<User>
            {
                new User { Name = "Alice", Roles = new List<string> { "Admin",
            },
                new User { Name = "Bob", Roles = new List<string> { "Admin" }
                new User { Name = "Charlie", Roles = new List<string> { "Admin"
            } }

            };
            bool allUsersAreAdmins = users.All(user => user.Roles.Contains("Admin"));
            if (allUsersAreAdmins)
            {
                Console.WriteLine("All Users are Admin");
                // Perform sensitive operation
            }
            else
            {

```

```

        Console.WriteLine("All Users are not Admin");
    }

    Console.ReadKey();
}

public class User
{
    public string Name { get; set; }
    public List<string> Roles { get; set; }
}
}

```

Output: All Users are Admin

Data Validation Across a Collection

Imagine you're processing a collection of order data, and you need to ensure that all orders are from the current year to proceed with a batch operation.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var orders = new List<Order>
            {
                new Order { OrderId = 1, Date = new DateTime(2024, 1, 10) },
                new Order { OrderId = 2, Date = new DateTime(2024, 2, 15) },
                new Order { OrderId = 3, Date = new DateTime(2024, 3, 20) }
            };
            bool allOrdersFromCurrentYear = orders.All(order => order.Date.Year ==
DateTime.Now.Year);
            if (allOrdersFromCurrentYear)
            {
                Console.WriteLine("Validation Success");
                // Proceed with batch operation
            }
            else
            {
                Console.WriteLine("Validation Failed");
            }

            Console.ReadKey();
        }
    }
}

```

```

public class Order
{
    public int OrderId { get; set; }
    public DateTime Date { get; set; }
}
}

```

Output: Validation Success

Product Inventory Check

Consider a scenario where you need to check if all products in a customer's shopping cart are in stock before allowing them to proceed to checkout.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var cartItems = new List<CartItem>
            {
                new CartItem { ProductId = 1, Quantity = 2 },
                new CartItem { ProductId = 2, Quantity = 1 },
                new CartItem { ProductId = 3, Quantity = 5 }
            };
            var inventory = new Dictionary<int, int>
            {
                { 1, 5 }, // Product ID 1 has 5 items in stock
                { 2, 2 }, // Product ID 2 has 2 items in stock
                { 3, 5 }  // Product ID 3 has 5 items in stock
            };
            bool allItemsInStock = cartItems.All(item => inventory[item.ProductId] >= item.Quantity);
            if (allItemsInStock)
            {
                // Allow the user to proceed to checkout
                Console.WriteLine("Proceed for Checkout");
            }
            else
            {
                Console.WriteLine("Some Product Out of Stock");
            }

            Console.ReadKey();
        }
    }
}

```

```

public class CartItem
{
    public int ProductId { get; set; }
    public int Quantity { get; set; }
}
}

```

Output: Proceed for Checkout

Validating Input Data

Suppose you have a list of email addresses entered by users in a form, and you need to validate that all email addresses are in the correct format before processing them.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var emails = new List<string> { "user1@example.com", "user2@exampl
"user3@example.org" };
            bool allEmailsValid = emails.All(email => Regex.IsMatch(email,
@"^[^@\s]+@[^@\s]+\.[^@\s]+$"));
            if (allEmailsValid)
            {
                Console.WriteLine("All Emails are Valid");
                // Process the emails
            }
            else
            {
                Console.WriteLine("All Emails are Not Valid");
            }

            Console.ReadKey();
        }
    }
}

```

Output: All Emails are Valid

Ensuring Consistent Data Across Multiple Fields

In a dataset of customer information, you might want to ensure that all customers have both a non-empty name and a valid postal code before running a marketing campaign.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var customers = new List<Customer>
            {
                new Customer { Name = "Alice", PostalCode = "12345" },
                new Customer { Name = "Bob", PostalCode = "23456" },
                new Customer { Name = "Charlie", PostalCode = string.Empty }
            };
            bool allDataIsValid = customers.All(c => !string.IsNullOrEmpty(c.PostalCode));
            if (allDataIsValid)
            {
                // Launch marketing campaign
                Console.WriteLine("Launch Marketing Campaign");
            }
            else
            {
                Console.WriteLine("Do Not Launch a Marketing Campaign");
            }

            Console.ReadKey();
        }
    }

    public class Customer
    {
        public string Name { get; set; }
        public string PostalCode { get; set; }
    }
}

```

Output: Do Not Launch a Marketing Campaign

In the next article, I will discuss the [LINQ Any Method in C#](#) with Examples. In this article, I try to explain the Linq ALL Method in C# with Examples. I hope you understand the need and use of the LINQ All Method in C# with Examples.